

Amaliy ish № 10

Mavzu: Favqulodda holatlar bilan ishlash. (2 soat)

Ishning maqsadi: Talabalarga sinf a'zolariga kirish uchun ruxsat modifikatorlari haqida tushuncha berish. Sinf a'zolari kirishni tashkil etish.

Kerakli asbob-uskunalar va jihozlar: fan bo'yicha O'UM, dasturlash bo'yicha topshiriqlar to'plami, kompyuter, videoproektor.

Masalaning qo'yilishi: Murakkab toifadagi maslalani yechish. Ruxsat modifikatorlaridan foydalanish.

Qisqacha nazariy ma'lumotlar.

C# tilida "holatlar" (states) deb nomlanadigan ma'lumotlar, dastur davomida o'zgaruvchan bo'lgan obyektlarga tegishli bilgilar yoki xolatlar. Bunday holatlar obyektning vazifalarini bajarishda va ma'lumotlarni saqlashda muhim rol o'ynayadi.

Quyidagi mavzular "holatlar" bilan ishlash mavzusida sizga ma'lumot beradi:

Obyektning holatini ifodalash uchun C# tilida "properties" (xususiyatlar)dan foydalanish. Xususiyatlar obyektning ma'lum bir holatini tutadi. Misol uchun, shaxs obyekti uchun "ism" va "yosh" xususiyatlari mavjud bo'lishi mumkin.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
Person person = new Person();
```

```
person.Name = "John";
```

```
person.Age = 25;
```

Obyektning holatini o'zgartirish uchun "method"lardan foydalanish. Methodlar obyektning vazifalarini bajarish uchun ishlatiladi. Misol uchun, shaxs obyekti "yoshni oshirish" va "yoshni kamaytirish" methodlari bo'lishi mumkin.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void IncreaseAge()
    {
        Age++;
    }

    public void DecreaseAge()
    {
        Age--;
    }
}
```

```
Person person = new Person();
```

```
person.Name = "John";
```

```
person.Age = 25;
```

```
person.IncreaseAge(); // Yoshni oshirish
```

```
person.DecreaseAge(); // Yoshni kamaytirish
```

Obyektning holatini tekshirish uchun "if" va "switch" ifodalaridan foydalanish. Bunda obyektning holatiga qarab qaror qabul qilinadi. Misol uchun, shaxs obyektida "o'qishga yaroqli" holatiga ega bo'lishi tekshirilishi mumkin.

```
class Person
```

```
{  
    public string Name { get; set; }  
    public bool IsStudent { get; set; }  
}
```

```
Person person = new Person();  
person.Name = "John";  
person.IsStudent = true;
```

```
if (person.IsStudent)  
{  
    Console.WriteLine(person.Name + " o'qishga yaroqli.");  
}  
else  
{  
    Console.WriteLine(person.Name + " o'qishga yaroqsiz.");  
}
```

Obyektlar orasida muloqotda holatlarni o'zgartirish uchun "event"lardan foydalanish. Bunda bir obyektning holati o'zgarganda boshqa obyektlarga xabar yetkazish mumkin. Misol uchun, tug'ilgan kuniga ozodlik bayrami kutilishini bildirishadigan "Tug'ilganKun" hodisasi bo'lishi mumkin.

```
class Person
```

```
{  
    public string Name { get; set; }  
    public event EventHandler Birthday;  
  
    protected virtual void OnBirthday()  
    {  
        Birthday?.Invoke(this, EventArgs.Empty);  
    }  
}
```

```
Person person = new Person();  
person.Name = "John";  
person.Birthday += (sender, e) =>  
{  
    Console.WriteLine(person.Name + " tug'ilgan kunigacha ozodlik bayrami kutilmoqda!");  
};
```

```
person.OnBirthday(); // Tug'ilgan kuniga ozodlik bayramini bildirish
```

Holatlar, obyektlarni dinamik va o'zgaruvchan qilishga imkon beradi. Ularning yordamida obyektlar ma'lum bir vazifani bajarish, ma'lumotlarni saqlash va o'zgartirish mumkin bo'ladi.

Umid qilamanki, bu ma'lumotlar sizga C# tilida holatlar bilan ishlash mavzusida tushuncha berishda yordam beradi. Agar boshqa savollar yoki C# tilida "holatlar" (states) deb nomlanadigan ma'lumotlar, dastur davomida o'zgaruvchan bo'lgan obyektlarga tegishli bilgilar yoki xolatlar. Bunday holatlar obyektning vazifalarini bajarishda va ma'lumotlarni saqlashda muhim rol o'ynaydi.

Quyidagi mavzular "holatlar" bilan ishlash mavzusida sizga ma'lumot beradi:

Obyektning holatini ifodalash uchun C# tilida "properties" (xususiyatlar)dan foydalanish. Xususiyatlar obyektning ma'lum bir holatini tutadi. Misol uchun, shaxs obyekti uchun "ism" va "yosh" xususiyatlari mavjud bo'lishi mumkin.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

```
Person person = new Person();
person.Name = "John";
person.Age = 25;
```

Obyektning holatini o'zgartirish uchun "method"lardan foydalanish. Methodlar obyektning vazifalarini bajarish uchun ishlatiladi. Misol uchun, shaxs obyektida "yoshni oshirish" va "yoshni kamaytirish" methodlari bo'lishi mumkin.

```
class Person
{
    public string Name { get; set; }
    public int Age { get; set; }

    public void IncreaseAge()
    {
        Age++;
    }

    public void DecreaseAge()
    {
        Age--;
    }
}
```

```
Person person = new Person();
person.Name = "John";
person.Age = 25;
```

```
person.IncreaseAge(); // Yoshni oshirish
person.DecreaseAge(); // Yoshni kamaytirish
```

Obyektning holatini tekshirish uchun "if" va "switch" ifodalaridan foydalanish. Bunda obyektning holatiga qarab qaror qabul qilinadi. Misol uchun, shaxs obyektida "o'qishga yaroqli" holatiga ega bo'lishi tekshirilishi mumkin.

```
class Person
{
    public string Name { get; set; }
    public bool IsStudent { get; set; }
}
```

```
Person person = new Person();
person.Name = "John";
person.IsStudent = true;
```

```
if (person.IsStudent)
{
```

```

        Console.WriteLine(person.Name + " o'qishga yaroqli.");
    }
    else
    {
        Console.WriteLine(person.Name + " o'qishga yaroqsiz.");
    }

```

Obyektlar orasida muloqotda holatlarni o'zgartirish uchun "event"lardan foydalanish. Bunda bir obyektning holati o'zgarganda boshqa obyektlarga xabar yetkazish mumkin. Misol uchun, tug'ilgan kuniga ozodlik bayrami kutilishini bildirishadigan "Tug'ilganKun" hodisasi bo'lishi mumkin.

```

class Person
{
    public string Name { get; set; }
    public event EventHandler Birthday;

    protected virtual void OnBirthday()
    {
        Birthday?.Invoke(this, EventArgs.Empty);
    }
}

```

```

Person person = new Person();
person.Name = "John";
person.Birthday += (sender, e) =>
{
    Console.WriteLine(person.Name + " tug'ilgan kunigacha ozodlik bayrami kutilmoqda!");
};

```

person.OnBirthday(); // Tug'ilgan kuniga ozodlik bayramini bildirish

Holatlar, obyektlarni dinamik va o'zgaruvchan qilishga imkon beradi. Ularning yordamida obyektlar ma'lum bir vazifani bajarish, ma'lumotlarni saqlash va o'zgartirish mumkin bo'ladi.

Topshiriqlar

1. Bir "BankAccount" nomli klass yarating. Bu klassda "balance" (saldov) xususiyati bo'lsin, va uni 0 ga tenglang. Klassda "Deposit" va "Withdraw" nomli methodlar bo'lsin, "Deposit" methodi hisobga pul qo'shish uchun ishlatilsin, "Withdraw" methodi esa hisobdan pul yechish uchun ishlatilsin. "Withdraw" methodi hisobdagi mablagani "balance" ni tashqariga chiqarib berishga imkon berishin.
2. Bir "Car" nomli klass yarating. Bu klassda "speed" (tezlik) xususiyati bo'lsin. "Accelerate" va "Brake" nomli methodlar yarating. "Accelerate" methodi mashinani tezlashtirish uchun ishlatilsin va "Brake" methodi esa mashinani tezligini pasaytirish uchun ishlatilsin.
3. Bir "Student" nomli klass yarating. Bu klassda "name" (ism) va "grade" (bahol) xususiyatlari bo'lsin. "Grade" xususiyati 0 dan 100 gacha bo'lishi kerak. Klassda "SetName" va "SetGrade" nomli methodlar bo'lsin, bu methodlar orqali "name" va "grade" xususiyatlarini o'zgartirishingiz mumkin bo'lsin.
4. Bir "Rectangle" nomli klass yarating. Bu klassda "width" (kenglik) va "height" (balandlik) xususiyatlari bo'lsin. Klassda "CalculateArea" nomli method yarating, bu method orqali to'g'ri burchakli to'rtburchakning yuzini hisoblash mumkin bo'lsin.
5. Bir "Employee" nomli klass yarating. Bu klassda "name" (ism), "position" (lavozim) va "salary" (maosh) xususiyatlari bo'lsin. Klassda "Promote" va "IncreaseSalary" nomli methodlar bo'lsin, "Promote" methodi ishchini lavozimini o'zgartirish uchun, "IncreaseSalary" methodi esa ishchining maoshini oshirish uchun ishlatilsin.

6. Bir "Product" nomli klass yarating. Bu klassda "name" (nomi), "price" (narxi) va "quantity" (miqdori) xususiyatlari bo'lsin. Klassda "GetTotalPrice" nomli method yarating, bu method orqali mahsulotning umumiy narxini hisoblash mumkin bo'lsin.
7. Bir "Circle" nomli klass yarating. Bu klassda "radius" (radius) xususiyati bo'lsin. Klassda "CalculateCircumference" va "CalculateArea" nomli methodlar bo'lsin, "CalculateCircumference" methodi doiraning perimetri, "CalculateArea" methodi esa doiraning yuzini hisoblash uchun ishlatilsin.
8. Bir "Book" nomli klass yarating. Bu klassda "title" (nomi), "author" (muallif) va "year" (yili) xususiyatlari bo'lsin. Klassda "GetBookInfo" nomli method yarating, bu method orqali kitob haqida to'liq ma'lumotni olish mumkin bo'lsin.
9. Bir "Bank" nomli klass yarating. Bu klassda "name" (nomi) va "branches" (filiallar) xususiyatlari bo'lsin. "branches" xususiyati "BankBranch" nomli boshqa klass obyektlarini saqlab turishi kerak bo'lsin. Klassda "AddBranch" va "RemoveBranch" nomli methodlar bo'lsin, "AddBranch" methodi bankga yangi filial qo'shish uchun, "RemoveBranch" methodi esa filialni o'Bank' nomli klassning davomi:
10. Bir "Bank" nomli klass yarating. Bu klassda "name" (nomi) va "branches" (filiallar) xususiyatlari bo'lsin. "branches" xususiyati "BankBranch" nomli boshqa klass obyektlarini saqlab turishi kerak bo'lsin. Klassda "AddBranch" va "RemoveBranch" nomli methodlar bo'lsin, "AddBranch" methodi bankga yangi filial qo'shish uchun, "RemoveBranch" methodi esa filialni o'chirish uchun ishlatilsin.
11. Bir "BankBranch" nomli klass yarating. Bu klassda "name" (nomi), "location" (joylashuvi) va "employees" (ishchilar) xususiyatlari bo'lsin. "employees" xususiyati "Employee" nomli boshqa klass obyektlarini saqlab turishi kerak bo'lsin. Klassda "AddEmployee" va "RemoveEmployee" nomli methodlar bo'lsin, "AddEmployee" methodi filialga yangi ishchi qo'shish uchun, "RemoveEmployee" methodi esa ishchini filialdan o'chirish uchun ishlatilsin.